# Hangman Web Game: Development Report

**CSE 4095 - Agentic Programming** | Individual Assignment

## Development process and AI assistance

I built my Hangman web game in eight phases instead of trying to generate the entire application at once. I started by reviewing the assignment, original specification, TODO file, and instructor transcript. I narrowed the project to the required Python game engine, Flask server, and browser client, leaving out the terminal client, bots, scoring, and leaderboard. I used ChatGPT as my AI coding assistant to plan and implement one part at a time. After each phase, I reviewed the changes, ran tests or tried the features myself, and documented the results before moving forward.

## Implementation

I wrote the core rules in **game.py**, separate from Flask: checking letter input, tracking correct and incorrect guesses, displaying the masked word, allowing six incorrect guesses, and determining wins and losses. I created **word_manager.py** to load and randomly select Easy, Medium, or Hard words. My Flask application in **server.py** handles **POST /api/games**, **GET /api/games/<game_id>**, and **POST /api/games/<game_id>/guess**. It stores games by unique IDs and returns JSON without exposing the answer during an active game. The browser uses HTML, CSS, and JavaScript, including **fetch()**, to show the game and submit guesses; the server controls the actual rules.

## Testing and debugging

I added automated tests for the game engine, word selection, REST API, and complete API-driven games. I ran the tests locally and confirmed they passed. I also manually tested the three difficulty levels, correct and incorrect guesses, the Hangman drawing, win/loss messages, and starting another game. In Chrome Developer Tools, I inspected a letter-guess HTTP request and its JSON response. I checked that the masked pattern updated and that the hidden word was not sent while the game was still in progress.

## Challenges and solutions

One problem came from Git: I accidentally ran **git init** inside my Hangman folder even though I already had a cloned repository. Git could not find the remote or branch I expected. I identified the extra nested repository and removed only its new **.git** directory, keeping the project files intact. Another issue appeared during deployment setup when I entered **server.py** as Render's Root Directory. I corrected it to **projects/hangman**, which is the actual folder within my repository.

## Deployment

I pushed the project to the **projects** branch of my existing GitHub repository and deployed it to Render. I used **pip install -r requirements.txt** as the build command and **gunicorn --bind 0.0.0.0:$PORT --workers 1 server:app** as the start command. I used one worker because games are stored in memory. Render reported a successful deployment, and the public website was reported accessible at **hangman-web-game-gjar.onrender.com**. I still need to capture my final hosted gameplay demonstration for submission.

## Representative AI prompts

• Revise the instructor's Hangman TODO into a phased plan for the browser and Flask server only. Do not write the whole application yet.

• Implement just the core Hangman engine in Python and write focused tests. Keep Flask and the browser out of this phase.

• Add integration tests and a browser/Developer Tools checklist without changing the working gameplay.

## Reflection

The biggest thing I learned was how to guide AI through smaller steps instead of accepting a complete application without checking it. Keeping the game engine separate from Flask made testing easier, and inspecting network requests helped me understand how JavaScript, HTTP, JSON, and Python work together. I also learned that an application working locally is different from configuring it correctly for cloud hosting. Testing and understanding the generated code were just as important as generating it.